

Checklist

Front End System Design Cheatsheet

Architecture

- Server-side rendering
- Client-side rendering
- Incremental static regeneration
- Serverless
- Edge computing
- Server components

State management

- Lazy loading
- Flux architecture
- Normalized stores

Back end APIs

- RESTful
- GraphQL
- Pagination (offset vs cursor)
- Authentication
- Authorization

Real-time updates

- Short polling
- Long polling
- Server-sent events
- WebSockets

Component APIs

- Theming config
- Events handlers
- Render props
- Composition

Networking techniques

- Batching
- Retries w/ exponential backoff + jitter
- Debounce & Throttling
- Optimistic updates
- Timeout
- Out-of-order / Race conditions
- Offline usage
- Caching + expiry

Performance

- Bundle splitting, lazy loading, code splitting (Route-based, interaction-based, visibility-based)
- Tree shaking
- Preloading and prefetching (JS, data)
- List virtualization
- Compression (code, media)
- Optimize loading sequences
- Prioritize above-the-fold
- Debounce & Throttling
- Optimistic updates
- Caching / Memoizing
- Memory leaks
- DOM changes: Batching and minimizing
- Reducing reflows and repaints
- Adaptive loading
- Core web vitals

Images

- CDN
- Format: WebP, SVG for icons
- Priority-based loading, lazy loading via loading="lazy"
- Alt text
- Responsive images: srcset, <picture>
- <link rel="preload">
- Adaptive loading (network speed, screen size)

Accessibility

- Keyboard interactions and shortcuts
- WAI ARIA roles, states, properties: Semantic HTML, img alt text, ARIA attributes, role attribute
- Headings hierarchy
- Focus management: focus indicators, tab order
- Visual: Color contrast, font size

SEO

- <title>, <meta>
- Sitemap
- JSON structured data
- Semantic markup
- Heading hierarchy
- SSR or SSG (Pre-generate pages)

User experience

- States: Loading, error, success, empty, offline
- Error handling: Ignore, retry, or display?
- Infinite scrolling
- Long strings
- Mobile-friendliness

Security

- HTTPS
- XSS
- CSRF
- CORS
- CSP

i18n

- <html lang>, hreflang attribute
- Right-to-left
- Use template strings
- Dates, currencies, numbers formatting
- Avoid text in images
- Input method editors (IMEs)

Common patterns

- Infinite scrolling
- Event sourcing
- Reducer pattern / Flux architecture
- Undo/redo
- Media streaming
- Conflict resolution
- Offline-first/usage

1 / 1

–

+



By Patricia Tani

Based on Yangshun Tay's Frontend System Design Checklist

Prompt for explanation:

Can you describe what <topic> is in nextjs and provide some examples of where it would be used and how it would be implemented, why you would choose this over other alternatives, and a tldr

Can you describe what Flux Architecture for Topic is in nextjs and provide some examples of where it would be used and how it would be implemented, why you would choose this over other alternatives, when you'd not want to use it, some real world apps that would use this technique for a feature, and a tldr

Architecture

☒ ~~Server-side rendering~~

SSR means Nextjs server generates HTML on **each request**, using live data. Requests are what happen every time your browser navigates to a URL. Eg. user visits page. **Browser asks server** for content. SSR pre-renders HTML on each request then hydrates on the client for interactivity. Faster and can help w/ **SEO and initial content**, but may increase TTFB (Time To First Byte), isn't always faster than SSG or cached responses.

Use Async for api requests. Async pauses and waits for data before continuing. Regular functions don't wait for that data. Await waits for a promise to finish, always needed in async. Reading file, API, waiting for user input is async.

Examples:

- `getServerSideProps()` (page router), `fetch(api)`, `data = await res.json()`, return data
- Server components / route handlers (app router)

Decide on whether pages are built per-request (SSR) or at build time (SSG) or rendered only on client (CSR).

Use when SEO matters, data changed frequently, content personalized per user, and page must fetch fresh data on each request.

SSR vs SSG:

- SSG pre-renders pages at build time (fast, cacheable), but bad if data changes frequently
- SSR wins for frequently updated content

☒ ~~Client-side rendering~~

Used for dashboards with live data, user-specific content (after login, as data only makes sense after user logins in), interactive UIs, and admin panels (SEO doesn't matter). Not for public facing data.

Browser downloads a minimal HTML page and the JavaScript needed for the page. **Javascript updates DOM** and renders page. User will experience slight delay before they can see full page. Takes longer bc JavaScript needs time to download, parse, and execute. Navigating to other pages is faster after loaded. Js can re-render parts without requiring full page refresh.

Impacts SEO as search engine crawlers may not execute javascript. Also worse for devices that are slower. Can use loading UI with Suspense to show loading indicator.

Examples:

- `useEffect()`, `fetchdata = async function, await fetch(api)`. Has catch e
- Recommended to use `useSWR` from `swr` for data fetching library, eg `const { data, error, isLoading } = useSWR('https://api.example.com/data', fetcher)`

☒ ~~Incremental static regeneration~~

Incremental = regenerates automatically.

Enables you to:

- Update static content without rebuilding entire site, builds at build time.
- You can update them later in the background using **revalidate** after a certain time.
- Reduce server load by serving **pre rendered static pages** for most requests.
- Proper cache control headers are automatically added to pages.

Cache control looks like this:

Cache-Control: public, max-age=0, s-maxage=10, stale-while-revalidate

This tells the browser and CDN (like vercel) how to handle caching of pages. Allows users to get fast pages (from CDN cache), update after a short time (based on **revalidate**)

☒ ~~Serverless~~

Used for API routes, middleware edge functions.

Serverless in Next.js = You write logic (SSR, API, Middleware), and it runs on-demand in tiny, auto-scaling cloud functions.

You don't manage your own server, and code runs in small auto-scaling functions called Lambdas. You only pay when your code is actually running. Eg. you only pay for a food truck that appears exactly when someone orders, vs a whole restaurant.

When you deploy to Vercel, Netlify, or AWS etc, certain parts, esp API routes and server-side functions, get deployed as serverless functions.

Examples:

- `getServerSideProps` -> only runs on every request, dynamic SSR.
- API routes (`/pages/api/*.ts`). Acts like backend logic (eg form handler, auth).
- Middleware edge functions, **runs before request hits page**.

Implementation:

✓ 1. API Routes

```
ts                                                                    Copy Edit

// /pages/api/hello.ts
export default function handler(req, res) {
  res.status(200).json({ message: "Hello from serverless!" });
}
```

- This is automatically deployed as a serverless function.

✓ 2. Server-side Rendering (SSR)

```
ts                                                                    Copy Edit

// /pages/posts.tsx
export async function getServerSideProps() {
  const data = await fetchData();
  return { props: { data } };
}
```

✓ 3. Middleware

```
ts                                                                    Copy Edit

// middleware.ts
export function middleware(request) {
  // Logic like auth, redirects, etc.
}
```

- This runs as an **Edge Function** — super fast, globally distributed.

Where is it used?

- Dynamic pages (eg User Dashboards, search results)
- APIs (eg Contact Forms, Stripe payments, login/signup)
- Scheduled tasks (with tools like Vercen Cron), refreshing data every hour
- Auth logic (validating sessions or JWTs)

- Webhooks (Stripe, Github, etc pinging your app).

☑ ~~Edge computing~~

Used in Middleware, edge API routes for User Auth tokens, geolocation, etc.

Edge computing means running code as close as possible to the user — on servers distributed around the world (called "edge locations").

It reduces latency (delay) and makes responses super fast, especially important for things like personalization, authentication, and redirects.

In Next.js, edge functions are a lightweight version of serverless functions that run at the **network edge** instead of a **centralized server location**.

Used for:

- Authentication (validate cookies or tokens before showing page)
- AB Testing / Personalization (shows user-specific content based on region or device)
- Redirects / Rewrites (smart routing based on request details)
- Rate limiting (blocking abusive requests)
- Geolocation logic (location aware pages)

Middleware runs **before request hits page** or API route. Great for **redirects, auth checks, geolocation**.

```
// middleware.ts
```

```
import { NextResponse } from 'next/server';
```

Real-World Example

Auth Middleware:

```
ts Copy Edit

// middleware.ts
import { NextResponse } from 'next/server';

export function middleware(request) {
  const token = request.cookies.get('token');
  if (!token) {
    return NextResponse.redirect('/login');
  }
  return NextResponse.next();
}
```

✓ Runs near the user, so login checks feel instant — no round trip to a central server.

☐ Server components

Components that:

- Only run on server
- Do not include Javascript in bundle
- Fetch data directly from database or API
- Improve performance and reduce JavaScript on client

Benefits:

- Safe to access secrets (call DBs or use private tokens)
- Less hydration work for browser

Use Cases:

- Blog post pages (fetch markdown or content from DB on server)
- Product Listings (pull data from CMS or database)
- Static/dynamic page shells (server HTML content quickly)
- Auth-protected pages (read cookies/session from server safely)

You can nest client components inside server components. Doesn't use browser APIs or React hooks.

State Management

☑ Lazy loading

Components or resources only loaded when needed (eg come into viewport or accessed by user).

Improves initial load time by reducing JS needed to be loaded initially.

Use Cases:

- Non-essential components like Modals, Charts, or Heavy Components
- Images (only when about to come into view eg Galleries, long lists)
- Routes (code splitting), pages that aren't initially visible to users

Implementation:

- Dynamic imports with `React.lazy()` and `next/dynamic`. EG: components lazily. Example:



```
tsx                                                                    Copy Edit

// Import a component lazily
import dynamic from 'next/dynamic';

const HeavyComponent = dynamic(() => import('../components/HeavyComponent'),
  loading: () => <p>Loading...</p>, // Optional loading state
);

export default function Page() {
  return (
    <div>
      <h1>Welcome to my page!</h1>
      <HeavyComponent />
    </div>
  );
}
```

- Images are lazy loaded by default automatically.
- Lazy loading routes, good for inside a dashboard:

```
tsx                                                                    Copy Edit

import dynamic from 'next/dynamic';

const AboutPage = dynamic(() => import('./About'), {
  ssr: false, // Optional: Disable server-side rendering
});

export default function HomePage() {
  return <AboutPage />;
}
```

☑ Flux architecture

Application architecture pattern that focused on **unidirectional data flow to manage state** in React applications. “Flux” = flow or stream of data that flows in single, predictable direction. Pattern that contrasts with 2-way data binding like in AngularJS

Uses Actions, a Dispatcher, and Stores to centralize state management.

[Action] → [Dispatcher] → [Store] → [View]

1. **Actions:** Objects that send data from the application to the dispatcher. They contain a **type** (indicating the action), and an optional **payload** with data.
2. **Dispatcher:** Central hub that receives actions and forwards them to stores. Ensures updates happen in consistent and predictable order.
3. **Stores:** Hold application state and logic. **Respond to actions and emit updates** when state changes. UI listens for these changes to update the view.
4. **Views:** React components that store updates and re-render when state changes. One-way from store to view.

Use Cases:

- Large apps when multiple components need to share and react to state changes.
- Handle Global State (eg user authentication, UI states)

Implementation:

- Redux, or useReducer which makes similar unidirectional flow to Flux.

1. **Creating an Action:** Actions are dispatched when a user interacts with the app.

```
ts                                                                    Copy Edit

// actions.ts
export const increment = () => ({
  type: 'INCREMENT',
});

export const decrement = () => ({
  type: 'DECREMENT',
});
```

2. **Creating a Reducer (Store):** A reducer handles actions and updates the state.

```
ts                                                                    Copy Edit

// reducer.ts
interface State {
  count: number;
}

type Action = { type: 'INCREMENT' } | { type: 'DECREMENT' };

const initialState: State = { count: 0 };

export const counterReducer = (state: State, action: Action): State => {
  switch (action.type) {
    case 'INCREMENT':
      return { count: state.count + 1 };
    case 'DECREMENT':
      return { count: state.count - 1 };
    default:
      return state;
  }
};
```

3. **Using the Reducer in a Component (View):** The component listens to the store (via the reducer) and dispatches actions.

```
tsx                                                                    Copy Edit

import { useReducer } from 'react';
import { increment, decrement } from './actions';
import { counterReducer } from './reducer';

export default function Counter() {
  const [state, dispatch] = useReducer(counterReducer, { count: 0 });

  return (
    <div>
      <p>Count: {state.count}</p>
      <button onClick={() => dispatch(increment())}>Increment</button>
      <button onClick={() => dispatch(decrement())}>Decrement</button>
    </div>
  );
}
```

☒ Normalized stores

Structuring global state to **avoid deeply nested or duplicated data** - by using a flat, relational-like format.

They are ways to organize your data like in a database (eg. Entities stored in flat structure using IDs as Keys), and relationships between entities are stored by referencing those IDs.

Avoids deeply nested objects, and redundant data across the state tree.

Common when using **Redux**, or any large scale state management system.

Why use:

- Makes updating/deleting entities easy
- Improves performance when selecting/updating slices of state
- Prevents UI inconsistencies due to duplicate state copies.
- Helps manage **relational data** (eg. posts with authors, comments with users)

Use Cases:

- Blog or forum (eg posts, authors, comments)
- An e-commerce app (eg. products, categories, cart items)
- Dashboard with complex nested data (eg. projects, users, tasks)

Implementation:

1. Define Normalized Structure via interface/type. Eg

```
interface NormalizedState {  
  posts: {  
    byId: Record<string, Post>;  
    allIds: string[];  
  };  
  authors: {  
    byId: Record<string, Author>;  
    allIds: string[];  
  };  
}  
  
type Post = {  
  id: string;  
  title: string;  
  content: string;  
  authorId: string;  
};
```

2. Use Redux to manage the state by creating an initial state, reducers with actions, and store.
3. Use **selectors** to denormalize eg

```
// selectors.ts  
import { RootState } from './store';  
  
export const selectPostWithAuthor = (postId: string) => (state: RootState) => {  
  const post = state.posts.byId[postId];  
  const author = state.authors.byId[post.authorId];  
  return { ...post, author };  
};
```

4. Use in Next.js component

tsx

Copy

Edit

```
import { useSelector } from 'react-redux';
import { selectPostWithAuthor } from '../selectors';

export default function PostItem({ postId }: { postId: string }) {
  const post = useSelector(selectPostWithAuthor(postId));

  return (
    <div>
      <h2>{post.title}</h2>
      <p>by {post.author.name}</p>
      <p>{post.content}</p>
    </div>
  );
}
```

Back End APIs

☐ RESTful

If **App Router**, go with server actions (flux architecture). If **Pages Router**, stick to API routes (REST).

Use API routes for when you need server-side endpoints that can be accessed from anywhere eg await openai api. Use Actions like in Flux when you're managing client-side state or local component logic.

Creating or consuming REST APIs that follow standard HTTP methods (GET, POST, PUT, DELETE etc) and use clear URL structures.

You build them using **API routes**, and use them on the Client side with fetch() or tools like **Axios**.

RESTful (Representational State Transfer) describes set of design principles for building web services:

- Stateless: Each request contains all info needed (no server-side session).
- Resource-based: URL paths represent resources (eg /api/users/1)
- Standard HTTP methods mentioned above

Examples:

- Building a blog, e-comm, dashboard where the frontend interacts with dynamic backend data.
- Creating custom backend endpoints (eg /api/login, /api/posts).

□ GraphQL

GraphQL is a query language and runtime for APIs.

Alternative to REST for querying data. Instead of multiple endpoints (eg /api/users, /api/posts), GraphQL uses **one endpoint** where clients can ask exactly the data they need.

- Great for dynamic UIs and component-driven apps
- Apollo, urql, or Relay
- When you want **fewer requests** and precise control over data.

Differences from REST:

- Single endpoint (/graphql) vs multiple (/api/post, /api/bruh)
- Client determines data shape, vs fixed server-defined data shape.
- No over/under-fetching
- For versioning, REST uses versioned endpoints while GraphQL's schema evolves

Implementation:

1. Client sends GraphQL queries from frontend
2. Server exposes GraphQL API

Why GraphQL over REST

Example **Github** easily chooses GraphQL:

- Github, as a developer dashboard, needs different data for each different component (RepoCard needs repos, UserBadge needs user data, PullRequestList needs prs).
- With REST, this would need 5 endpoint calls. With Graph, one call is needed in a big query. Since data is relational and nested, you can do everything in 1 query

□ Pagination (offset vs cursor)

Pagination = **splitting large data sets into smaller chunks**. Can be used for infinite scroll, but Cursor is better for that.

Two diff types:

1. **Offset pagination:** skip N items, then take X items.
Use when **total items is predictable** or small, don't care if data changes mid-scroll, for admin panels, product lists, blogs. Has a page tracker and a limit. EG fetching github repos.
2. **Cursor pagination:** use reference (ID, timestamp) to fetch next batch.
Use **pointer** of last seen item like id or timestamp, saves performance by not skipping and is more consistent as new rows won't shift pagination.

Offset is simple and common, Cursor better for performance and real-time (eg social feeds).

Examples:

- Instagram uses Cursor pagination via timestamps
- Amazon uses Offset pagination (pages 1-N)

- Twitter uses Cursor (realtime feed)
- CMS Admin uses Offset (easy pagination logic)

□ Authentication

Process of **verifying identity** of user, typically through usernames/passwords or OAuth, and ensuring user has permission to certain resources. Usually has Provider wrapping to provide auth state.

Common Types of Auth:

1. Session-based (server-side, stores in cookies)
2. JWT (JSON Web Tokens), (token-based, stateless, usually stored in cookies or localStorage)
3. OAuth (using third-party services like Google or Github)

Clientside vs Serverside:

1. Client verifies directly on frontend using tokens
2. Server checks sessions or API keys

Libraries: NextAuth.js, or API routes for custom auth logic.

Session-based vs JWT Auth

- Storage is stored on the server (cookies, database), vs JWT that stores on client (localStorage, cookies).
- Scalability - session needs server-side session store, JWT is stateless and scales easily
- State - Keeps session on server, easy to invalidate vs JWT gives more control on clientside
- Session is more secure by default (server validation), JWT might have potential issues with token storage (XSS cross-site scripting).

□ Authorization

Process of ensuring that an authenticated user has the right permissions to access specific resources or actions. Happens after authentication, after you verify the user's identity and then check what they're allowed to do.

Types:

- Role-based authorization: Admin, user, guest, based on these roles, different actions or pages are allowed (eg Discord roles).
- Access control: Permissions for specific API routes, pages, or components based on user's role

When to use:

- Protect routes and pages based on user roles

- Protect API routes (eg only authenticated users can access certain data)

Implementation:

- **Pages Router:** Check user role during **SSR** via **getServerSideProps** to fetch from session or token, then check user's role.
- **App Router:** use **Server Components** (`cookies()` / `headers()`), Middleware ([NOT RECOMMENDED HERE'S WHY](#)), and/or route handlers

```
// pages/admin.tsx
import { GetServerSideProps } from 'next';
import { getSession } from 'next-auth/react';

const AdminPage = ({ user }) => {
  if (!user || user.role !== 'admin') {
    return <p>You do not have permission to access this page.</p>;
  }

  return (
    <div>
      <h1>Admin Dashboard</h1>
      { /* Admin content */ }
    </div>
  );
};

export const getServerSideProps: GetServerSideProps = async ({ req }) => {
  const session = await getSession({ req });

  if (!session || session.user.role !== 'admin') {
    return { redirect: { destination: '/', permanent: false } };
  }

  return {
    props: {
      user: session.user,
    },
  };
};
```

-
- How it works: inside `getServerSideProps`, `getSession` fetches the session data. If the session doesn't exist or user isn't an admin, they get redirected.

2. Protect API Routes Based on Roles:

```
ts                                                                    Copy Edit

// pages/api/protectedRoute.ts
import { getSession } from 'next-auth/react';

export default async function handler(req, res) {
  const session = await getSession({ req });

  if (!session || session.user.role !== 'admin') {
    return res.status(403).json({ error: 'You are not authorized to access this route' });
  }

  // Authorized users can access the resource
  res.status(200).json({ data: 'This is protected data' });
}
```

App Router implementation (3 ways):

1. Server Components run on the server, so you can safely read cookies or headers

directly.

```
tsx Copy code

// app/dashboard/page.tsx
import { cookies } from "next/headers";

export default async function DashboardPage() {
  const cookieStore = cookies();
  const session = cookieStore.get("session_token");

  if (!session) {
    // You could redirect unauthenticated users
    return <p>Unauthorized. Please log in.</p>;
  }

  // Fetch user info from your backend
  const res = await fetch(`${process.env.API_URL}/me`, {
    headers: { Cookie: `session_token=${session.value}` },
  });

  const user = await res.json();

  if (user.role !== "admin") {
    return <p>Forbidden. You don't have access.</p>;
  }

  return <div>Welcome, {user.name}! This is your admin dashboard.</div>;
}
```

Use this for **per-page role checks** to keep sensitive logic on the server.

2. Middleware ([NOT RECOMMENDED HERE'S WHY](#)) (Edge runtime)

Runs **before** your request hits a route. Great for protecting **groups of routes** (eg /admin).

Use this for global route protection & **redirecting unauthed users early**, before rendering.

ts

 Copy code

```
// middleware.ts
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(req: NextRequest) {
  const token = req.cookies.get("session_token")?.value;

  if (!token) {
    return NextResponse.redirect(new URL("/login", req.url));
  }

  // Example: decode token to check role
  const user = decodeJwt(token); // pseudo-code

  if (req.nextUrl.pathname.startsWith("/admin") && user.role !== "admin")
    return NextResponse.redirect(new URL("/forbidden", req.url));
}

return NextResponse.next();
}

// Protect only admin routes
export const config = {
  matcher: ["/admin/:path*"],
};
```

3. **Route Handlers** (API-like endpoints)

Lets you enforce auth at the **API level**.

ts

 Copy code

```
// app/api/admin/data/route.ts
import { NextResponse } from "next/server";
import { cookies } from "next/headers";

export async function GET() {
  const session = cookies().get("session_token");
  if (!session) {
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }

  const user = await validateSession(session.value);

  if (user.role !== "admin") {
    return NextResponse.json({ error: "Forbidden" }, { status: 403 });
  }

  // Return protected data
  return NextResponse.json({ secret: "admin-only-data" });
}
```

Real-time Updates

Feature	Short Polling	Long Polling	WebSockets
Connection Type	Frequent new requests	Persistent request until data is ready	Persistent, bidirectional connection
Latency	Depends on interval	Low (almost instant after data is ready)	Instant
Efficiency	Inefficient if polling too often	Efficient, responds only with new data	Very efficient
Complexity	Simple	Moderate	More complex
Use Case	News feed, low-priority updates	Notifications, chat apps, dashboards	Multiplayer games, live collaboration apps

☒ Short polling

Client repeatedly makes requests to the server at regular intervals (eg every few secs) to check for updates.

Simple to implement but can be inefficient, esp with high frequency polling or large numbers of clients.

Unlike **long polling** or **Websockets**, short polling sends requests at fixed intervals, regardless of whether data has changed, leading to **wasted resources**.

How it works:

1. Client makes HTTP request to server (eg /api/check-updates).
2. Server responds with current state of data.
3. After receiving response, client waits for 5-10 secs to make another request.
4. Repeats infinitely so client has most up-to-date information, although there may be delay based on polling interval.

When to use:

- Real-time updates are needed, but **low traffic** and **low-frequency of changes** are expected.
- **Doesn't require connection between client and server**, making it easier to scale.
- Don't need low-latency communication like Websockets.
- Server resources are limited, and you don't need persistent connection.

Examples:

1. News feed that updates every few secs with new articles.
2. Chat app where messages are periodically fetched from server
3. Live score app that updates scores every 10sec.

Implementation:

- setInterval in useEffect to call the api route, then cleans up interval upon unmount.

☐ Long polling

HTTP-based technique. Client sends request to server, and server **holds connection open** until it has new data (or times out). Once a response is received, the client re-initiates the request.

More efficient than short polling for **low-to-medium frequency** updates and doesn't require Websockets.

When to use:

- Real-time notifications or chat messages
- Live updates for dashboards (eg order tracking, stock prices)
- Websockets are overkill
- Want more efficient alternative to short polling but don't need persistent two-way connection

Implementation:

1. Create API route that holds the connection.
In a real app, this could be hooked into a database, pub/sub system, or async event source.

```

ts
Copy Edit

// pages/api/long-poll.ts
import { NextApiRequest, NextApiResponse } from 'next';

export default async function handler(req: NextApiRequest, res: NextApiResponse) {
  // Simulate waiting for new data
  const waitForData = () => {
    new Promise((resolve) => {
      const timeout = setTimeout(() => {
        resolve({ message: 'New data available!' });
      }, 10000); // simulate data ready in 10s (you can link this to DB or events)
    });
  };

  const data = await waitForData();

  res.status(200).json(data);
}

```

2. Client-side: Create a loop to continuously poll

```

tsx
Copy Edit

// components/LongPoller.tsx
import { useEffect, useState } from 'react';

export default function LongPoller() {
  const [message, setMessage] = useState<string>('');

  const poll = async () => {
    try {
      const res = await fetch('/api/long-poll');
      const data = await res.json();
      setMessage(data.message);
      poll(); // immediately start next request
    } catch (err) {
      console.error('Polling error:', err);
      setTimeout(poll, 3000); // retry after delay if error
    }
  };

  useEffect(() => {
    poll(); // start polling on mount
  }, []);

  return (
    <div>
      <h2>Real-Time Updates</h2>
      <p>{message}</p>
    </div>
  );
}

```

Why use Long Polling over alternatives?

- More efficient than short polling
- Low latency - client gets data almost immediately after it becomes available
- Works everywhere - uses HTTP, no special server setup needed unlike Websockets

Cons:

- Involves keeping connections open - could strain server with many clients

- Not true bidirectional (client always initiates)

Real world example:

1. Support ticket dashboard that shows new tickets in realtime to admins
 - a. Each admin client sends long polling request to /api/new-tickets
 - b. Server waits until new ticket is created, then responds
 - c. Client receives the new ticket, renders it, then immediately sends new request

☐ Server-sent events

SSE is built in browser API that creates persistent HTTP connection, where server keeps sending data over time (as events), using a **one-way stream**. Supported natively by **EventSource** in the browser.

Great for simple realtime feeds, notifications, or live updates. Easier than websockets, more efficient than polling, but only one-way (server -> client).

When Would You Use SSE in Next.js?

- Live dashboards (e.g., stock price updates)
- Notifications (e.g., “new comment” alerts)
- Server logs or status monitors
- Streaming AI responses (like OpenAI-style chats)
- Chat apps (with some caveats)

When would you use SSE over polling or websockets?

- You only need server -> client updates.
- Want something simpler than websockets
- You're ok with HTTP only

☐ WebSockets

Two-way communication between client and server for a persistent connection, making them ideal for real-time apps like chat, live games, collab tools, and financial dashboards.

Integrate using third-party server (eg socket.io, ws, or external WebSocket service like Ably or Pusher) along Nextjs.

Websockets let both sides **send messages at any time**. Super low latency and very scalable for real-time interaction.

Use cases:

- Live chat
- Multiplayer games
- Collaborative editors

- Crypto dashboards for realtime updates like stock prices, crypto
- Live shopping eg bids
- Livestream chat

Implementation

1. Use websocket server (like socket.io)

Since Next.js (esp App router) isn't optimized for persistent connections natively, you usually

 - Run a **custom Websocket server** (in same project or separately)
 - Or use hosted service (like Pusher, Ably, Supabase Realtime, or Liveblocks).

Example using socket.io in Nextjs

🔗 Setup WebSocket server (socket.ts)

```
ts
// lib/socket.ts
import { Server as HTTPServer } from 'http';
import { Server as SocketIOServer } from 'socket.io';

let io: SocketIOServer | null = null;

export const setupSocket = (server: HTTPServer) => {
  if (io) return io; // singleton

  io = new SocketIOServer(server, {
    path: "/api/socket",
    cors: { origin: "*" },
  });

  io.on("connection", (socket) => {
    console.log("New client connected");

    socket.on("send-message", (msg) => {
      console.log("Received:", msg);
      io?.emit("receive-message", msg); // broadcast
    });

    socket.on("disconnect", () => {
      console.log("Client disconnected");
    });
  });

  return io;
};
```

2. Add server-side hook (server.ts or middleware.ts).

Real-world Examples:

1. Slack/Discord - realtime messaging & presence
2. Figma - collaborative design changes
3. TradingView - live stock/crypto pricing updates
4. Kahoot - Realtime quiz responses
5. Twitch - Live chat & stream data

Component APIs

❑ Theming config

A config that defines a centralized, consistent set of design tokens (like colors, fonts, spacing, etc) that your components use - often thru a UI library (eg Tailwind, ShadCN, Chakra UI, Radix, etc). Allows for dynamic themes (light/dark), design consistency, and component-level customization across Next.js app.

❑ Events handlers

Event Handlers are functions that respond to user interactions like clicking a button, typing an input, or submitting a form. They're the same in React, but can be enhanced in nextjs via API routes, client-server interaction, and server actions (in app router).

Examples:

- onClick, onChange, onSubmit

Client-side example:

1. Button Click Handler (Client-side)

```
tsx Copy Edit  
  
// app/page.tsx or a client component  
'use client'  
  
export default function Home() {  
  const handleClick = () => {  
    alert('Button clicked!')  
  }  
  
  return <button onClick={handleClick}>Click Me</button>  
}
```

Form submission with API call


```
// app/contact/page.tsx (Client Component)
'use client'

const submitForm = async (e: React.FormEvent<HTMLFormElement>) => {
  e.preventDefault()
  const res = await fetch('/api/contact', {
    method: 'POST',
    body: JSON.stringify({ message: 'Hello' }),
  })
  const data = await res.json()
  console.log(data)
}

<form onSubmit={submitForm}>
  <button type="submit">Send</button>
</form>
```

Optimistic Updates

3. Optimistic UI Updates

```
jsx

const likePost = async () => {
  setLiked(true); // optimistic update
  await fetch('/api/like', { method: 'POST' });
};
```

Used in: Social media apps (Instagram, Twitter), collaboration tools (Trello).

Real world examples:

- Airbnb - filtering search results dynamically
- Facebook - Like buttons, comment submissions
- Shopify/Amazon - Add to card w/o reloading the page
- Notion/Google docs - Keyboard shortcuts and autosave

☒ Render props

Render props are a React pattern where a component uses a function as a prop to determine what to render. It allows code reuse between components in a flexible way without needing HOCs or hooks.

Example:

Basic Example

```
jsx Copy Edit

// MouseTracker.js
import { useState } from 'react';

const MouseTracker = ({ render }) => {
  const [position, setPosition] = useState({ x: 0, y: 0 });

  const handleMouseMove = (e) => {
    setPosition({ x: e.clientX, y: e.clientY });
  };

  return (
    <div onMouseMove={handleMouseMove} style={{ height: '100vh' }}>
      {render(position)}
    </div>
  );
};

export default MouseTracker;
```

Usage:

```
jsx Copy Edit

// page.jsx or component in Next.js
import MouseTracker from './MouseTracker';

export default function Page() {
  return (
    <MouseTracker
      render={({ x, y }) => (
        <h1>
          Mouse is at ({x}, {y})
        </h1>
      )}
    />
  );
}
```

Basically just create a function called like `MouseTracker` and use it as the parent passing the child props as `render` to determine the content. Therefore the parent has control over the result. It is a lot more flexible than hardcoding what a component should display.

A `render prop` is a function passed as a prop to the component so that the component can delegate the rendering logic to the caller.

☑ Composition

Practice of combining multiple smaller components to create more complex UIs instead of relying on big, monolithic ones.

Think of it in contrast to inheritance or prop drilling, it's more aligned with React's functional, declarative style.

Where is it used?

- Share layout or behaviour between components
- Customize components without rewriting them
- Avoid prop hell
- Create flexible, reusable UI primitives

Examples

- Layouts (`<Layout> <Sidebar/> <MainContent/> </Layout>`)
- UI libraries using children (think `<Card><CardHeader /><CardContent /></Card>`)
- Passing render props or slots (custom behaviour per child)
- Feature toggles, themes, permissions

It typically uses `children` as `React.ReactNode` and is a wrapper.

2. Component with Slots

```
tsx                                                                    Copy Edit

// components/Card.tsx
export function Card({ children }: { children: React.ReactNode }) {
  return <div className="rounded shadow p-4">{children}</div>;
}

// Usage
<Card>
  <h2>Title</h2>
  <p>Body text goes here.</p>
</Card>
```

3. Using Composition for Customizable UI

4. Advanced: Composable Feature Wrappers

```
tsx                                                                    Copy Edit

const AuthWrapper = ({ children }: { children: React.ReactNode }) => {
  const user = useUser();
  return user ? <{children}> : <LoginPrompt />;
};

<AuthWrapper>
  <ProtectedComponent />
</AuthWrapper>
```

Networking Techniques

☒ Batching

Combining multiple requests into a single network call to reduce round trips between client and server. This improves performance, reduces latency, and **lowers the load** on both the frontend and backend.

In NextJS, esp with React Server Components, App Router, and Server Actions, batching can be leveraged to optimize API calls and data fetching by grouping multiple fetches into one - either manually or automatically.

Why it matters

- Each network request comes with overhead (DNS lookup, TLS handshake, etc)
- If you're fetching multiple resources at one (eg user profile, settings, posts) doing that individually for each can be expensive.
- Batching reduces these into one, improving page load and responsiveness by a lot.

How to implement

1. Manual batching

Create an API route or server action that accepts multiple queries and returns all needed data in a **single response**.

```
ts                                                                    Copy Edit

// app/api/batch-data/route.ts
export async function GET(req: Request) {
  const [profile, posts, settings] = await Promise.all([
    getUserProfile(),
    getUserPosts(),
    getUserSettings(),
  ]);

  return Response.json({ profile, posts, settings });
}
```

```
ts                                                                    Copy Edit

// client-side call
const res = await fetch("/api/batch-data");
const { profile, posts, settings } = await res.json();
```

2. Batching with server components

When using server components, multiple `await` calls inside one server function (or layout/page) are **automatically batched** on the server.

```
tsx                                                                    Copy Edit

// app/dashboard/page.tsx
export default async function Dashboard() {
  const [profile, posts] = await Promise.all([
    getUserProfile(), // runs on the server
    getUserPosts(),
  ]);

  return (
    <div>
      <UserProfile data={profile} />
      <UserPosts data={posts} />
    </div>
  );
}
```

Next.js will batch the execution and reduce unnecessary client-to-server chatter.

3. You can also **batch Server Actions** into one Server Action, so the browser only sends one request to the server

```
// app/actions.ts
"use server";
export async function batchUpdate({ profile, settings }: any) {
  await updateProfile(profile);
  await updateSettings(settings);
}
```

☒ Retries w/ exponential backoff + jitter

Retries = trying an operation again after it fails

Exponential backoff = Wait longer between each retry (e.g., 1s → 2s → 4s → 8s...)

Jitter = Add randomness to avoid thundering herd problems (eg everyone retrying at the same time)

Together, they form a robust pattern to handle **temporary errors** like network hiccups, rate limits, or flaky APIs.

Why use it:

Since Nextjs often relies on remote APIs eg Supabase, REST APIs, they might

1. Rate limit requests
2. Occasionally time out
3. Fail transiently but succeed on retry

Instead of blowing up the user's experience on first failure, you can **retry intelligently** giving the system a chance to recover.

How to implement:

It's a technique you implement manually in your fetch logic or with a helper lib.

Example:

```
ts
// lib/Utils/retryFetch.ts
export async function retryFetch(url: string, options = {}, retries = 5) {
  let attempt = 0;

  while (attempt < retries) {
    try {
      const res = await fetch(url, options);
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      return res;
    } catch (err) {
      attempt++;
      const backoff = Math.pow(2, attempt) * 100; // exponential
      const jitter = Math.random() * 100; // add jitter
      const delay = backoff + jitter;

      if (attempt === retries) throw err;
      await new Promise((r) => setTimeout(r, delay));
    }
  }
}
```

Client Component implementation:

✓ Using in a Next.js Client Component

```
tsx
'use client';
import { useEffect, useState } from 'react';
import { retryFetch } from '@lib/Utils/retryFetch';

export default function Profile() {
  const [data, setData] = useState(null);

  useEffect(() => {
    retryFetch('/api/user')
      .then((res) => res.json())
      .then(setData)
      .catch((e) => console.error('Failed to fetch after retries:', e));
  }, []);

  if (!data) return <p>Loading...</p>;
  return <div>Welcome, {data.name}</div>;
}
```

☐ Debouncing & throttling

Debouncing is **waiting until the user has stopped triggering an event for a certain amount of time** before you run a function.

If they keep triggering it (like typing), you keep resetting the timer.

Used for search inputs since you don't want to fire an API request every keystroke, you want to wait until they pause typing (eg 300ms), then fetch results.

Throttling means **limiting the number of times a function can run in a given time window**, even if the event fires many times.

Eg. handling scroll events or window resizing. You don't want to update UI every pixel moved - you update, say, once every 500ms max.

Debounce examples:

- Search input that calls an API
- Auto-saving a form while typing

Throttling examples:

- Handling scroll animations (eg sticky headers)
- Recalculating layout based on window resize

☒ ~~Optimistic updates~~

Optimistic updates **immediately update the UI** as if the server action succeeded - **before** you actually get confirmation from the server.

"Let's pretend the action worked instantly, and we'll clean up later if the server disagrees."

Why?

- Speed. No lag, no waiting. Makes your app feel blazing fast and super responsive to the user.

Where would you use it?

- Like button on a social media post
- Adding/removing items from a shopping cart
- Submitting a comment on a blog post
- Saving settings (eg toggles)

Where it's used irl:

- Instagram: liking a photo
- Twitter: liking a tweet, retweeting
- Trello: moving cards around
- Discord: editing/sending/deleting messages

TLDR; use it for non-critical, low-risk actions.

Do not use it on actions that can't easily be undone because it's hard to rollback safely.

❑ Timeout

Setting a **maximum time limit** for requests like API calls, database queries, or remote fetching (eg `fetch()`). If response not in time, request is **aborted** or handled as failure.

Used in combo with:

- AbortController (native)
- Timeout wrappers
- Middleware
- Serverless timeouts

Where you'll use it:

- Server-side data fetching (`getServerSideProps`, `getStaticProps`)
 - Prevents SSR from hanging indefinitely if API is slow
- Client-side Fetches -> for UX and error handling (fallback UI)
- API routes
- Edge & Middleware (strict latency requirements)

Implement with AbortController (recommended)

1. Using AbortController (Recommended)

```
ts
// app/api/products/route.ts

export async function GET() {
  const controller = new AbortController();
  const timeout = setTimeout(() => controller.abort(), 5000); // 5 seconds

  try {
    const res = await fetch('https://api.example.com/products', {
      signal: controller.signal,
    });

    const data = await res.json();
    return Response.json(data);
  } catch (error) {
    if (error.name === 'AbortError') {
      return new Response('Request timed out', { status: 504 });
    }
    return new Response('Internal server error', { status: 500 });
  } finally {
    clearTimeout(timeout);
  }
}
```

Real world examples:

- E-Commerce (SSR) `getServerSideProps` timeout ensures stuck product API doesn't block whole page
- Finance Dashboard (times out slow stock price APIs and falls back to cached data)
- Social Media - avoid freeze timeline - use spinner/retry instead

❑ Out-of-order / Race conditions

Multiple network requests are sent but responses come back in wrong order, causing stale/incorrect data to be rendered/stored.

Out of order: data may be displayed as soon as ready, regardless of when other requests complete (non-blocking)

Race condition: bug when 2 or more async requests update same state, and later response overrides earlier.

Often happens in **client-side data fetching** when

- Users **rapidly interact** with search inputs, dropdowns, or filters
- Navigation triggers **multiple async requests**
- **Streaming or incremental rendering** is used, but data changes during rendering

When this happens:

- User typing in search bar
- Component fetches, but unmounted before response arrives
- Query params changing rapidly
- App router with server components, lead to mismatched hydration

IRL examples:

- Comments appear later than video on Youtube
- Google docs collaboration updates
- Twitter Timeline loads tweets while sidebar immediately available

How to Fix:

- AbortController: cancel stale fetches
- Request IDs: Only apply latest response if ID matches.
- React Suspense + streaming
- Server Components

☐ Offline usage

Allows app to function without live internet connection, by caching or storing data locally and syncing later. Falls under PWA techniques and resilient networking.

- If user loses network, can still see content (from cache)
- If take actions (like submitting form), actions are queued and synced once network restored.

In Nextjs, can store cached pages offline.

Get offline data via IndexedDB

- Works offline since read/write to localStorage

Why use:

- offline mode prevents white screens with flaky network
 - Users can continue workflows w/o interruption
 - Faster performance via cache

Implementation strats:

- Service workers with next-pwa
- Local storage/indexedDB
- Background sync api
- React query has it built in

Examples:

- Spotify's songs that u can just play w/o wifi
- Pinterest they gained 40% engagement once implementing

☐ Caching + expiry

Caching = storing copy of data (eg pages, api responses, assets) so asset can serve faster w/o refetching every time

Expiry = setting a “time to live” TTL or invalidation rule so cached data automatically refreshes

In Nextjs, can happen at multiple layers:

- Browser cache (controlled by HTTP headers)
- CDN/Edge cache (vercel edge, cloudflare)
- Server-side cache (fetch cache in nextjs, isr)
- Client-side cache (react query, swr)

Examples in Nextjs:

1. Static page with revalidation (ISR), cached at build/first req, revalidated every 60s

```
tsx Copy code

// app/posts/[id]/page.tsx
export async function generateStaticParams() {
  return [{ id: "1" }, { id: "2" }];
}

export default async function Post({ params }: { params: { id: string } }) {
  const res = await fetch(`https://api.example.com/posts/${params.id}`, {
    next: { revalidate: 60 }, // cache + expire every 60s
  });
  const post = await res.json();
  return <h1>{post.title}</h1>;
}
```

• Cached at build/first request

2. API fetch with revalidation, products cached server-side, every 5 min fresh data refreshed

```
tsx Copy code

// app/page.tsx
export default async function Page() {
  const res = await fetch("https://api.example.com/products", {
    next: { revalidate: 300 } // 5 minutes expiry
  });
  const products = await res.json();
  return <pre>{JSON.stringify(products, null, 2)}</pre>;
}
```

3. Client-side SWR (stale while revalidate)
 - shows cached data immediately
 - refreshes in background after expiry

```
tsx Copy code

"use client";
import useSWR from "swr";

const fetcher = (url: string) => fetch(url).then(res => res.json());

export default function Profile() {
  const { data, error } = useSWR("/api/user", fetcher, {
    refreshInterval: 60000, // revalidate every 60s
    dedupingInterval: 5000 // avoid duplicate calls
  });

  if (error) return <div>Failed to load</div>;
  if (!data) return <div>Loading...</div>;
  return <div>Hello {data.name}</div>;
}
```

4. Cache control headers

```
tsx Copy code

// pages/api/data.js
export default function handler(req, res) {
  res.setHeader("Cache-Control", "s-maxage=60, stale-while-revalidate");
  res.json({ time: Date.now() });
}
```

- Cached for 60s at edge/cdn
- During revalid, stale data can still serve until new data arrives

Why use?

- Performance: faster load times, reduced server load
- Freshness - no stale data
- Handle high traffic w/o hammering apis
- Customizable expiry time depending on data volatility

Alternatives:

- Always live fetch -> guarantees fresh data, but slower and costly
- Infinite cache (no expiry), super fast but risk incorrect/stale data
- Manual invalidation -> powerful but complex (need to purge cache on db write)

Examples:

- Timeline tweets cached for seconds
- Amazon Product info cached for mins, prices revalidated periodically
- News Articles cached for short TTL
- Netflix Recommendations cached with expiry

Performance

❑ Bundle splitting, lazy loading, code splitting [Route-based, interaction-based, visibility-based]

1. **Bundle Splitting:** breaking JS bundle into smaller pieces to improve load time since users only need to download code needed for current page/feature.

How in Nextjs: **automatic by default** (each page /pages or /app becomes its own chunk)

Shared code (like React, Nextjs runtime, or libraries imported into multiple places) is split into a separate vendor bundle.

2. **Lazy loading:** Loading components **only when needed**, instead of upfront. Reduces initial page load, improves performance on first paint

How in nextjs:

```
tsx Copy code

import dynamic from 'next/dynamic';

const Chart = dynamic(() => import('../components/Chart'), { ssr: false });

export default function Dashboard() {
  return (
    <div>
      <h1>Dashboard</h1>
      <Chart /> {/* Only loads when rendered */}
    </div>
  );
}
```

Dynamic imports with ssr: false, like for a chart library or map widget thats heavy but not critical to page load

Why use Dynamic imports with ssr = true vs regular server component

- use when u **reuse component multiple times** so it **auto caches** and page loads faster on page changes

- Cuz it forces it into its own JS bundle chunk that is shared with all ur pages
- Prevent bloated initial page bundles too, if import static, js bundle can get huge, each section gets own chunk
- Browser may load all immediately but still improves cache reuse & makes incremental updates smaller

Why NOT to use:

- More network requests cuz each dynamic import creates separate JS chunk.
- On client, hydration requires downloading multiple files vs 1 large bundle
- Components only used once have **no caching benefit**
- Hydration can be delayed for critical UI **“will flash”** even if ssr: true, hurts TTI
- Longer build times for webpack/turbo
- Complicates debugging (stack traces are harder to trace across chunks)

3. **Code Splitting Types:** Next.js automatically handles route-based splitting, but you can implement **interaction-based** (eg click button to open modals) or **visibility-based** (loading something only when touching 100 pixels above it) for finer control.

- a) **Route-based Code Splitting:** each route gets its own bundle, /dashboard and /settings will load separate JS bundles. Prevents loading dashboard code when user only visiting settings.
- b) **Interaction-based Code Splitting:** Code only load after user interaction (eg button click)

Example:

```
tsx Copy code

const [Editor, setEditor] = useState<null | React.ComponentType>(null)

const loadEditor = async () => {
  const mod = await import('../components/Editor');
  setEditor(() => mod.default);
};

return (
  <div>
    <button onClick={loadEditor}>Open Editor</button>
    {Editor && <Editor />}
  </div>
);
```

Rich text editor or image editor loaded only when someone clicks Edit

- c) **Visibility-based Code Splitting:** Code load when component scroll into view

```
tsx Copy code

import dynamic from 'next/dynamic';
import { useInView } from 'react-intersection-observer';

const Reviews = dynamic(() => import('../components/Reviews'));

const Page = () => {
  const { ref, inView } = useInView();

  return (
    <div>
      <h1>Product Page</h1>
      <div ref={ref}>{inView && <Reviews />}</div>
    </div>
  );
};
```

Customer reviews or related products **below the fold**.

Why use:

- Smaller initial bundle -> faster load -> better core web vitals
- Heavy code doesn't block rendering
- Makes large apps manageable, prevents performance degradation

- SEO benefits from faster loads (improve ranking)

Alternatives:

- Preloading everything (bad for large apps)
- Server-only rendering (good for some cases but adds server load and latency)
- Micro-frontends (heavier complexity than needed)

Examples:

- Netflix lazy load video player ui after main vid starts
- Airbnb load map components only when user scrolls to listings

□ Tree shaking

Process of auto removing unused code from JS bundles at build time by importing specific parts/functions from libs. Term comes from 'shaking a tree' so only fruit u want remains - unused branches fall away.

In NextJS:

- Uses modern bundlers (webpack or turbopack) that support tree shaking via ESM (ECMAScript Modules).
- If you import a library but only use part of it, the bundler will remove unused exports.

Example 1: Importing specific functions

Bad (no tree shaking)

ts

Copy code

```
import _ from "lodash"; // imports ALL of lodash
const result = _.uniq([1, 2, 2, 3]);
```

Better (tree shaking works)

ts

Copy code

```
import { uniq } from "lodash"; // only uniq is kept
const result = uniq([1, 2, 2, 3]);
```

Example 2: Component Libraries

Bad:

tsx

Copy code

```
import * as MUI from '@mui/material';
// using only <Button>, but you ship the ENTIRE MUI library
<MUI.Button>Click me</MUI.Button>
```

Good:

```
tsx Copy code

import { Button } from '@mui/material';
// only Button and its dependencies are bundled
<Button>Click me</Button>
```

Extra: does not apply to import * as React from "react" cuz react is not tree-shakable, need whole runtime. Only good if library has many independent functions (reduce 90% of code)

Example 3: Utility Libraries

Bad:

```
ts Copy code

import * as dateFns from "date-fns";
dateFns.format(new Date(), "yyyy-MM-dd");
```

Good:

```
ts Copy code

import { format } from "date-fns";
format(new Date(), "yyyy-MM-dd");
```

Example 4: NextJS

- Nextjs automatically tree-shakes unused features
- Eg if u dont use next/image, its code is removed from the client bundle
- Same with api routes, middleware, or next/font.

Questions u might have:

- does nextjs automatically tree shake unused components in components folder if not rendered on a page.tsx?
YES. if never import it in any page, nextjs will never bundle it
- What about wholeass libraries?
Yes only their used functions will be kept.
- What if the logic only execute during build time

It cuts all **dead code paths**

Works with process.env.NODE_ENV eg

```
ts Copy code

if (process.env.NODE_ENV !== "production") {
  console.log("dev-only logging");
}
```

Production build would strip this out.

- Unused image sizes (breakpoints) aren't generated unless requested
- **Server components shaken more aggressively** since never hit client bundle

❑ Preloading and prefetching (JS, data)

Most powerful performance features of Next.js bc navigation feels INSTANT.

Preloading: Load assets (JS, fonts, images, CSS) **ahead of time**, so when user needs, already in cache

- Usually done for critical (above-the-fold components).
- Browser mechanism: `<link rel="preload">`

Prefetching: Load assets or data for routes/pages that users might navigate to soon.

- Usually done in background
- Browser mechanism: `<link rel="prefetch">`

In Next.js, both handled automatically in smart ways, but u can customize

How it works in Next.js:

1. JS Bundle Prefetching

- Next.js auto prefetches the JS bundles of linked routes.
- Example:

```
tsx
import Link from 'next/link';

export default function Home() {
  return <Link href="/about">About Us</Link>;
}
```

Copy code

When `<Link>` visible in viewport, Next.js starts fetching `/about`'s JS bundle in the background. So when user clicks -> near instant navigation.

2. Data Prefetching

- In **NextJS App Router** (`/app`), use Server Components (async fetch) and Route Handlers; Next can prefetch the linked route's **Server Component payload** and code when `<Link/>` is in viewport
- With **NextJS Pages Router**, if route uses `fetch` or `getServerSideProps/getStaticProps`, Next.js can prefetch the data payload too.
- Controlled via:

```
tsx
<Link href="/dashboard" prefetch={true}>Dashboard</Link>
```

Copy code

- prefetch={true} (default): fetches both JS and data
- prefetch={false}: fetches only on navigation

3. Image Preloading

- Nextjs Image supports priority prop:

tsx

Copy code

```
<Image src="/hero.png" alt="Hero" width={1200} height={800} priority /
```

- Adds <link rel="preload"> so the hero image is downloaded early
- Improves Largest Contentful Paint (LCP)

4. Font Preloading

- Using next/font

tsx

Copy code

```
import { Inter } from 'next/font/google';
const inter = Inter({ subsets: ['latin'], display: 'swap' });
```

- Next auto injects <link rel="preload"> for font files
- Prevents FOUT/FOIT (flash of unstyled text)

Why use this over alternatives?

- Manual <link rel="preload">: u can handroll but easy to over-fetch
 - Lazy loading only -> delays navigation, slower UX
 - NextJS gives best balance, smart prefetch

Examples:

- Shopify product detail page bundles when user is browsing product cards -> instant page load
- Twitter prefetch tweet detail pages as soon as tweet link comes into view
- Vercel prefetches team/project data when hovering over links so nav feels seamless

Questions I had:

- Why would u choose to prefetch? How to determine what is important to prefetch
A: faster navigation, better UX. Conversion optimization (100-200 ms delay can ruin conversion). Mobile Optimization, network latency is higher so prefetch during idle times. Do for **Predictable User Behaviour** eg user browsing category page click product detail pages
- How does Nextjs handle these automatically? (it was answered above)
- Why would u want to do prefetch=false?
Save bandwidth on low-value routes, since prefetching downloads JS+data payloads for linked pages even if user never clicks

- Is the display: swap part important for prefetching?
- Do shopify/twitter/vercel have a specific way of prefetching that nextjs doesnt offer automatically like not <Links> or <Image priority/>?
- Do all Nextjs <Link/> have prefetch=true by default?
A: Yes, in page router it prefetches when Link in viewport, in App router its true as well but smart, waits till link visible + browser idle.
- Should I set prefetch=false on footer links that barely get clicks?
YES, cuz it wastes bandwidth + esp about pages w CMS content, saves mobile data as well. However, if lightweight, might as well keep on.
Eg Shopify doesnt prefetch legal links (Privacy, Terms). Stripe doesnt have Jobs, Notion links like Security/Help are prefetch false, but Templates/Pricing are prefetched.
- <a> doesnt have prefetch on by default

□ List virtualization

Also called windowing, is performance optimization technique for rendering large lists of data. Instead of rendering all items in list at once, which overwhelms browser's DOM and slows down rendering, only items **currently visible** in viewport (plus buffer above & below) are rendered. As user scrolls, offscreen items are replaced with newly visible ones.

In Nextjs, virtualization is not built in, but you can integrate libraries like react-window, react-virtualized, react-virtuoso.

Where it's used:

Essential when dealing w **large data sets** or **infinitely scrolling**.

Example use cases:

1. **Social media feeds** -> twitter's endless timeline or facebook's feed, only small slice of posts are rendered at a time.
2. **Ecomm product listings** -> amazon search results: tens of thousands of items, but only page worth is in the DOM.
3. **Chat apps** -> Slack, Discord, ChatGPT, but only viewport messages render.
4. **Data Tables & Dashboards** -> financial apps with huge spreadsheets (robinhood, google sheets).

How to Implement in Nextjs:

Basic implementation w react-window:

tsx

Copy code

```
// app/page.tsx or pages/index.tsx
"use client";

import { FixedSizeList as List } from "react-window";

const items = Array.from({ length: 100000 }, (_, i) => `Item ${i}`);

export default function HomePage() {
  return (
    <div style={{ height: "500px", width: "300px" }}>
      <List
        height={500}           // viewport height
        itemCount={items.length}
        itemSize={35}          // fixed row height
        width={300}
      >
        {({ index, style }) => (
          <div style={style}>
            {items[index]}
          </div>
        )}
      </List>
    </div>
  );
}
```

Why choose virtualization?

- Performance gains, DOM stays light - important for lists of 10,000+ items
- Smooth scrolling: reduces lag/jank when scrolling large lists
- Memory efficiency: DOM nodes aren't wasted on offscreen items
- SEO: can still server-render the initial viewport, while virtualizing client-side scrolling.

Alternatives:

- Pagination/load more button -> simpler & SEO friendly, but breaks seamless UX
- Infinite scrolling (w/o virtualization) -> eventually bloats DOM and hurts performance
- Skeleton loading -> improves perceived performance but doesn't solve DOM bloat

Questions:

- How to implement infinite scrolling with list virtualization like in reddit (i am guessing cursor-based pagination?)

A: need 2 layers:

- infinite scrolling (data loading), fetches more items as user scrolls down

(IntersectionObserver or load more trigger)

- List virtualization (rendering): renders only visible slice of list (react-window or smth) etc.

Just use react-window, react-virtuoso or whatever to do it

❑ Compression (code, media)

Compression reduces size of assets sent over network. Smaller files = faster downloads = faster page loads.

In Nextjs compression applies to:

- Code -> JS, CSS, HTML, JSON
- Media -> Images, fonts, audio, video

Nextjs (esp Vercel) auto handles some optimizations, but u can config extra compression strats for better Core Web Vitals and Lighthouse scores.

Types of Compression in Nexts:

1. Code compression:

- Gzip: default, widely supported, good baseline
- Brotli: more modern, 15-20% smaller files vs gzip, supported in most browsers
- Minification: removing whitespace, comments, dead code

Example: sending [main.js](#) bundle as 120KB gzip instead of 350KB raw.

2. Media compression:

- **Nextjs Image Optimization (next/image)**: automatically serves modern formats (webp, avif) when supported. resizes and compresses images on-the-fly
- **Video**: store in efficient codecs (H.265, VP9, AV1), stream instead of preloading entire files.
- **Fonts**: use next/font with subsetting -> only ship characters actually used.

Example: An image that's 2MB raw gets delivered as 200KB AVIF thumbnail.

Implementation with Next/Image:

```
tsx                                                                    Copy code
import Image from "next/image";

export default function ProductPage() {
  return (
    <Image
      src="/big-product.jpg"
      alt="Product"
      width={800}
      height={600}
      quality={75} // adjust compression
      priority // preload important images
    />
  );
}
```

- Serves optimized formats per browser, handles responsive sizes automatically.

Why choose compression:

- Faster load times -> better UX + SEO (google ranks faster pages higher)

- Reduced bandwidth costs -> important for media heavy apps
- Better performance on slow networks -> mobile users, developing regions

Alternatives:

- CDN-based optimization (cloudflare, akamai, fastly): similar results but need external setup
- Client-side lazy loading: higher performance perception but doesn't reduce transfer size
- Caching only: reduces downloads but doesn't shrink file sizes

Questions:

- So is doing breakpoint responsive sizes in ur next image like `fill sizes="(max-width: 768px) 100vw, (max-width: 1200px) 50vw, 400px"` better than simply `width={400}`?
Yes if image scales with viewport (fluid, 100vw, grid-based etc). `width={400}` always requests 400 no matter what screen size, wastes. `fill + sizes` tells browser to change based on viewport width.
No if rendered size never changes and use it for icons, avatars, logos, small UI assets that don't scale with layout.
- Why would u use CDN-based optimization instead of nextjs built in compression/image optimization?
 Instead of app server handling compression/optimization, CDN does it at edge. Global performance, closer to user, lower latency. Better caching. Offload work from server. More customization

□ Optimize loading sequences

Control **when and how** assets and code are loaded so your app feels fast.

If everything loads at once, users may see slow initial rendering (white screen). If critical resources load first, users see meaningful content quickly, even if secondary features load later.

Prioritize critical assets + Deferring non-critical ones = fast app

Techniques in Nextjs:

1. Code Splitting & Dynamic imports for components not needed immediately
2. Image priority/lazy loading for above/below fold stuff
3. Font optimization to eliminate flash of invisible text (FOIT)
4. Resource hints like preload/prefetch
5. Streaming + Suspense: send HTML progressively as data resolves

Why Use:

- Better Core Web Vitals (LCP, FID, CLS)

- Reduce JS cost by deferring heavy components

Alternatives:

- Monolithic loading (ship everything at once), simpler but hurts performance
- Etc described above

☐ Prioritize above-the-fold

Above the fold = part of page visible **without scrolling** when first load.

- For first impressions eg Hero, Navigation bar, logos, or featured content
- Optimize this to ensure users see meaningful content asap (improves LCP).

Techniques in Nextjs:

1. Prioritize Hero Images with <Image priority/>
2. Critical fonts with next/font
3. Defer non-essential scripts to prevent blocking rendering

☐ Debouncing & throttling

Performance patterns ([explained here](#)) to deal with **frequent user or system interactions** (scroll, resize, search input, api calls)

What is Debouncing?

Group a series of rapid events into single call after delay.

- Function returns only after event **stops firing** for X ms.
- Use when only want to react to final action

Throttling:

Allow function to run at most **once every X ms**, no matter how many times it fires.

- When you want regular updates but not too many eg scroll position or analytics dashboard.

Example: Throttled scroll event (like twitter infinite scroll)

```
tsx Copy code

"use client";

import { useEffect } from "react";
import throttle from "lodash.throttle";

export default function Feed() {
  useEffect(() => {
    const handleScroll = throttle(() => {
      if (window.innerHeight + window.scrollY >= document.body.offsetHeight) {
        console.log("Load more posts...");
      }
    }, 200); // run at most once every 200ms

    window.addEventListener("scroll", handleScroll);
    return () => window.removeEventListener("scroll", handleScroll);
  }, []);

  return <div style={{ height: "200vh" }}>Scroll me</div>;
}
```

- Logic runs every 200ms

Why use:

- Bandwidth saving, UX, performance

Alternatives:

- setTimeout -> weaker version of debouncing
- RequestAnimationFrame for scroll/resize -> better syncs with browser paint but doesn't limit frequency itself.
- Web Workers doesn't reduce triggers

Real world apps:

- Google Maps throttle map dragging + zoom
- Amazon debounce filters/search fields
- Reddit rebound search inputs
- Facebook throttle infinite scrolling feeds

☐ Optimistic updates

[Read about here:](#) Improve **user-perceived performance** rather than actual app performance. Normally, when user clicks "Add to Cart", UI would freeze until network roundtrip finishes. With optimistic updates, UI responds immediately, making app feel super fast even if backend slow.

- Improves First Input Responsiveness (FIR) and Interaction to Next Paint (INP) which are Core Web Vitals
- Reduce repaints & jank, instead of showing loading state, skip to final state.
- Reduce need for polling by assuming new state.

❑ Caching / Memoizing

Caching: storing prev fetched/processed **data** (server or client) so you don't recompute/refetch

Memoization: caching **function results** in memory (typically in client components) so repeated inputs give instant results w/o computation

Client-side memoization:

useMemo, useCallback, or libs like SWR/React Query cache data

Use when:

- Data doesn't change frequently (eg product catalog, blog posts, config values)
- Expensive computations need to be reused
- External APIs have latency/usage costs
- Large lists or complex UI would rerender unnecessarily

Examples:

- Avoid rerenders of child components with useMemo/useCallback

Implement in Nextjs:

1. Client-Side memoization

```
tsx Copy code

import { useMemo } from "react";

function ProductList({ products }) {
  // Memoize an expensive computation
  const sortedProducts = useMemo(() => {
    console.log("Sorting products..");
    return [...products].sort((a, b) => a.price - b.price);
  }, [products]);

  return (
    <ul>
      {sortedProducts.map((p) => (
        <li key={p.id}>{p.name} - ${p.price}</li>
      ))}
    </ul>
  );
}
```

Prevents re-sorting on every render unless products changes

2. Server-side Caching with fetch

```
tsx Copy code  
  
// Cached fetch request  
export default async function Page() {  
  const res = await fetch("https://api.example.com/products", {  
    next: { revalidate: 60 }, // cache for 60 seconds  
  });  
  const products = await res.json();  
  
  return <ProductList products={products} />;  
}
```

3. SWR (stale while revalidate) on client

```
tsx Copy code  
  
import useSWR from "swr";  
  
const fetcher = (url: string) => fetch(url).then((res) => res.json());  
  
function Profile() {  
  const { data, error } = useSWR("/api/user", fetcher, { revalidateOnFocus:  
  
    if (error) return "Failed to load";  
    if (!data) return "Loading...";  
  
    return <div>Welcome {data.name}</div>;  
}
```

Questions:

- What is difference between useMemo and useCallback?
A: useMemo cache result of computation, useCallback cache function itself (important only when ur **passing a function to memoized child**)
- When bad to use and why?
A: when data changes frequently, or critical high-integrity data for stock prices, healthcare, where stale cache can be dangerous

☐ Memory leaks

When application allocates memory but fails to release after no longer needed. Leads to increased memory usage, degraded performance, and crashes.

In Nextjs, memory leaks not about low-level memory management (like C++) but **dangling references** and **unmanaged resources** in components, hooks, or server code.

Common Causes:

1. **Unsubscribed event listeners** -> eg `window.addEventListener` without removing them on unmount
2. Uncleared Timers/intervals
3. Stale network requests (api calls completing after component unmount), caused by updating state on unmounted component
4. Server memory leaks (nextjs API routes/server actions) -> storing large objs in global vars, holding onto request/response objects after finishing, caching too aggressively w/o removal policy
5. Improper use of refs / closures -> retaining refs to DOM nodes, state, or large objects longer than needed

How to Prevent:

- always clean up effects in `useEffect` with `return () => { ... }`
- Use `AbortController` to cancel fetches on unmount
- Avoid global state bloat (keep caches scopes and manageable)
- Profile memory usage with Chrome DevTools -> Performance -> memory tab
- For server code, implement **cache eviction** (eg LRU aka least recently used strat) to avoid unnecessary objs

Real world apps that can be impacted:

- Slack/Discord w/ chat sessions running for hours -> must clear up API requests/observers
- Twitter/X -> infinite scroll feed with unmounted components
- Video players attach listeners for playback/resize
- Shopify with many filters, modals, and api calls -> leaks degrade checkout flow speed

Questions:

- How to detect in nextjs if you have a memory leaked stale network request (code example)?

A: since **most common memory leak source**, if u start fetch inside `useEffect` and try to `setState` after the component has unmounted, React **warns you** (in dev) and u leak memory in prod cuz references stick around.

Example of leaky pattern:

```
tsx                                                                    Copy code

useEffect(() => {
  fetch("/api/data")
    .then((res) => res.json())
    .then((data) => setData(data)); // ❌ may run after unmount
}, []);
```

Why is this considered leaky? If component unmounts before fetch is resolved, `setData` still tries to run but react doesnt have the component mounted. `setData` and associated data **stay alive** until GC -> memory leak risk

Look for this warning: “**Can't perform a React state update on an unmounted component.**”

Easier way is just use AbortController and set error message to console.warn(“Stale state”).

- How to keep track of all active event listeners in a page
This apparently records every listener that gets attached or removed:

```
js Copy code

(function () {
  const originalAdd = EventTarget.prototype.addEventListener;
  const originalRemove = EventTarget.prototype.removeEventListener;

  // Store active listeners in a Set
  window.__activeListeners = new Set();

  EventTarget.prototype.addEventListener = function (type, listener, options) {
    window.__activeListeners.add({ target: this, type, listener, options });
    originalAdd.call(this, type, listener, options);
  };

  EventTarget.prototype.removeEventListener = function (type, listener, options) {
    for (let l of window.__activeListeners) {
      if (l.target === this && l.type === type && l.listener === listener) {
        window.__activeListeners.delete(l);
        break;
      }
    }
    originalRemove.call(this, type, listener, options);
  };
})();
```

- Once ur done, type in devtools console: “window.__activeListeners”

❑ DOM changes: Batching and minimizing

DOM changes are expensive, updating DOM (adding/removing/reflowing/repainting elements) is one of the **slowest operations** on the browser.

Batching: group multiple DOM updates together instead of doing them one by one

Minimizing: reduce how many updates necessary in first place (only update what truly changed)

In Nextjs/React 18+, u don't manually touch the DOM bc React uses Virtual DOM.
React also automatically batches state updates, even across async boundaries

We still cause unnecessary rerenders -> must watch to minimize them

Where used:

1. Multiple state updates in React components
2. Large lists/tables -> minimize by rendering only visible items with virtualization, batching to avoid updating 1000+ DOM nodes individually

3. Form interactions -> combine mult form field updates into single render to avoid rerendering all fields when only one field changes
4. [Next.js](#) hydration -> React batches updates to avoid layout thrash (code force browser to recalculate layout/geometry in between DOM mutations)

How to Implement:

1. Rely on React 18's auto batching

```
tsx Copy code

"use client";
import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0);
  const [loading, setLoading] = useState(true);

  async function increment() {
    setLoading(true);
    await new Promise(r => setTimeout(r, 1000));
    setCount(c => c + 1);
    setLoading(false); //  batched with setCount
  }

  return <button onClick={increment}>{loading ? "...": count}</button>;
}
```

Both setCount and setLoading **trigger one render**, not 2.

2. useMemo/useCallback
3. Virtualize large DOM updates w/ react-window and methods described above
4. Server Components so they generate on server vs hydrate on client

IRL Examples:

- Twitter -> timelines update in batches vs repainting 1 tweet at a time
- Gmail -> large list of emails -> virtualization to minimize DOM
- Notion -> collab edits batched
- Amazon -> product filters don't reload whole page

Questions:

- Does react include useEffect setState with regular setState code during batching?

```

tsx Copy code

import { useEffect, useState } from "react";

export default function Demo() {
  const [count, setCount] = useState(0);
  const [loading, setLoading] = useState(false);

  useEffect(() => {
    setLoading(true);    // state update #1
    setCount(c => c + 1); // state update #2
    // In React 18 -> batched, triggers one render
  }, []);

  return (
    <p>
      Count: {count}, Loading: {loading ? "yes" : "no"}
    </p>
  );
}

```

This would be one render if React 18, two if React 17.

❑ Reducing reflows and repaints

Reflow (layout): when browser recalcs **positions and sizes** of elements. Triggered when you change things like element size, font, or DOM structure. (think of flow -> flows down when layout shifts)

Repaint: When the browser updates **pixels (colors, styles)** of elements, but layout doesn't change (eg background-color) (think paint -> color/styles)

Cost difference: Reflow more expensive than repaint because it can cascade down the DOM tree.

Where it would be used:

1. Interactive UI (form dashboards chats)
2. Infinite scroll feeds - avoid recalc layout for 100s of items
3. Animations/transitions - animate with **transforms**, not layout affecting properties
4. Responsive grids/lists - avoid triggering global reflows by batching style changes

How to reduce reflows/repaints w Nextjs:

1. Batch DOM updates

```
tsx                                                                    Copy code

// ❌ causes 2 reflows/repaints (separate renders)
setWidth(newWidth);
setHeight(newHeight);

// ✅ React 18 batches these → 1 render → fewer reflows
setDimensions({ width: newWidth, height: newHeight });
```

2. Use CSS transforms instead of layout changes

```
css                                                                    Copy code

/* ❌ triggers reflow */
.element {
  left: 100px;
  top: 50px;
}

/* ✅ only triggers repaint/composite (GPU-accelerated) */
.element {
  transform: translate(100px, 50px);
}
```

3. Virtualize Large lists eg using fixedsizelist from react-window

4. Minimize Layout Thrash (read + write together)

```
js                                                                    Copy code

// ❌ alternating reads/writes → thrash
el.style.width = "200px";
console.log(el.offsetHeight);
el.style.height = "100px";

// ✅ batch reads then writes
const h = el.offsetHeight;
el.style.width = "200px";
el.style.height = h + "px";
```

[Next.js](#) specific optimizations:

- Dynamic imports -> load noncrit components later -> fewer initial paints
- Server components -> heavy work server side -> smaller client payload
- NextJS Image -> avoid layout shift by reserving width/height of img

Questions:

- Can u give an example lets say u want to open a navbar dropdown and for it to animate its width going bigger or smaller, would u animate it with transform-translate or absolute width?

A: **use transform: scaleX** (but it stretches element), so inner content will stretch. To avoid, wrap content in another element, only scale container.

- **Can u give common sources of repaints/reflows that react noobs often do**
A: Animating layout properties eg width, height, top, left, margin, instead use transform translate/scale or opacity. Forgetting to set width/height on images. Forcing reads after writes:

js

```
element.style.width = "200px";           // write → invalidates layout
console.log(element.offsetHeight);       // read → forces reflow
```

Large lists without virtualization. Unnecessary state causing global rerenders eg all app state in one App component -> any change forces children to re-render -> DOM recalcs

^ fix this by splitting states, eg instead of 1 global “appState”, split into “authState”, “themeState”, “cartState”, etc, memoize, use Zustand, or external state managers (react query, swr)

☐ Adaptive loading

Performance optimization strat where **app adapts** to amount or type of resources it loads depending on user’s device capabilities, network conditions, or other runtime factors by adding/reducing features/content quality. Instead of serving the same heavy exp to everyone, **app “adapts” to lightweight for low-end devices and heavy for high-end devices.**

In Nextjs, means conditional rendering, code splitting, and selectively serving assets based on user context.

Examples of where it’d be used:

- 1. Images & Media:** smaller, compressed images or fewer animations to low-end devices eg next/image with sizes and quality props
- 2. Component rendering:** rendering simplified UI versions (eg static lists vs infinite scroll) on devices with limited sources, dynamically importing heavy libs like charts, maps, or edits only for capable devices.
- 3. Network Conditions:** if user on 2G/3G, load low-resolution video streams or defer secondary scripts
- 4. Features based on device memory:** use Network Information API (provides info ab users internet connection eg bandwidth, round-trip time) or Device Memory API (exposes an appropriate amount of device RAM)

How to implement in Nextjs:

1. Dynamic imports based on available deviceMemory

Example: Adaptive Loading with Device Memory

```
tsx Copy code

import { useEffect, useState } from "react";
import dynamic from "next/dynamic";

export default function Page() {
  const [Chart, setChart] = useState<React.ComponentType | null>(null);

  useEffect(() => {
    // Check device memory (in GB) - lower values = low-end devices
    if (navigator.deviceMemory && navigator.deviceMemory >= 4) {
      const DynamicChart = dynamic(() => import("../components/HeavyChart"
        ssr: false,
        loading: () => <p>Loading chart...</p>,
      ));
      setChart(() => DynamicChart);
    }
  }, []);

  return (
    <div>
      <h1>Dashboard</h1>
      {Chart ? <Chart /> : <p>Showing simplified stats for low-end devices</p>
    </div>
  );
}
```

👉 Here, the chart is **only loaded on devices with ≥4GB RAM**. Users with less memory get a simpler alternative.

2. Device/Network Checks (clientside)

```
tsx Copy code

useEffect(() => {
  if (navigator.connection && navigator.connection.effectiveType === "2g") {
    setLowDataMode(true);
  }
}, []);
```

3. Conditional Rendering

```
tsx Copy code

{isLowEndDevice ? <StaticList /> : <InfiniteScroll />}
```

4. Image Optimization

tsx

Copy code

```
<Image
  src="/hero.jpg"
  alt="Hero"
  width={1200}
  height={600}
  sizes="(max-width: 600px) 100vw, 50vw"
  quality={isLowEndDevice ? 40 : 80}
/>
```

Why not to use Adaptive Loading:

- Don't use for small apps w/o heavy features, adds unnecessary complexity
- Inconsistent UX, frustration if features "disappear" across devices/connections
- Don't use for SEO-critical content, crawlers won't see it
- Browser support, navigator.deviceMemory API is not supported everywhere

IRL Examples:

- Youtube has "Youtube Go" for low-bandwidth environments with lighter features
- Instagram Life/Facebook Lite for lower resource versions of slow networks
- Google search uses adaptive images and resource hints to prio fast loading on bad devices
- Ecomm sites load simpler product detail pages w/ reduced media on low-end devices

□ Core web vitals

Core Web Vitals = set of user-centered performance metrics defined by Google to measure real UX of web page. Directly impacts **SEO ranking**. Focus on **loading speed, interactivity, and visual stability**.

3 main metrics:

- Largest Contentful Paint (LCP) -> how quickly main content loads
- First Input Delay (FID) -> how quickly page responds to user interaction
- Cumulative Layout Shift (CLS) -> how stable page layout is while loading

In Nextjs, can measure and optimize these with built-in support (reportWebVitals) and framework features like next/image, code splitting, and automatic optimization.

Examples where used:

1. Image-heavy sites (eg e-comm, news), optimize hero images with next/image to reduce LCP
2. Interactive dashboards or SaaS apps -> reduce blocking scripts and hydration delays to improve FID
3. Dynamic content feeds -> prevent unexpected shifts with **reserved space** (CSS aspect-ratio, skeletons) to reduce CLS.

4. Marketing or landing pages

How to Implement in Nextjs:

1. Collect Core Web Vitals

In `pages/_app.tsx`:

```
tsx Copy code

export function reportWebVitals(metric) {
  console.log(metric) // { id, name, label, value }

  // Example: send to analytics
  if (metric.name === 'LCP' || metric.name === 'FID' || metric.name === 'CLS') {
    fetch('/analytics', {
      method: 'POST',
      body: JSON.stringify(metric),
    });
  }
}
```

This hooks into Next.js's built-in Web Vitals reporting.

2. Optimize each metric

LCP: use `next/image` for optimized images, use `priority` for above-the-fold content

FID: Split large JS bundles with dynamic imports, avoid long sync scripts on load

CLS: always set width/height (or aspect ratio) for images and ads, use `font-display: swap`. Font-display: swap

Why use Core Web Vitals:

- Backed by Google, used in SEO ranking, and reflect real UX
- Clear metrics with thresholds
- Built into Nextjs through `reportWebVitals` api, need to export special function shown above
- Alternatives like Lighthouse scores are useful, but **Core Web Vitals directly affect search visibility** and align with user-centric outcomes

When NOT to use Core Web Vitals:

- Internal-only (eg admin dashboard) that doesn't care about SEO
- In early prototypes/MVPs
- Audience has predictable networks eg Enterprise users use 90% high-end computers

Questions:

- How does google determine what is the lcp? Is it the biggest thing in the above fold area?

A: Yes, the **largest visible element in viewport** (above fold) to render on screen

- What are most common react noob mistakes that ruin core web vitals

A: Not using `next/image` instead using `<img>`, blocking fonts by adding Google fonts w/ `<link>` -> makes all text invisible until fonts load (bad LCP). Lazy loading everything even for hero image/CTA.

- What would optimized next/image images look like above vs below the fold? What props would they use like loading=lazy and stuff like that

A: Above the fold:

```
tsx
Copy code

<Image
  src="/hero.jpg"
  alt="Hero"
  width={1200}
  height={600}
  priority // ✅ preloads image for faster LCP
  fetchPriority="high" // ✅ hint for browsers (Next.js 13+)
  sizes="100vw" // ✅ responsive image sizing
  placeholder="blur" // optional: shows a blur-up placeholder
/>
```

- `priority` : Tells Next.js to preload this image. Should **only** be used for images in the initial viewport.
- `fetchPriority="high"` : Explicit browser hint to load it early.
- `sizes` : Ensures the correct responsive variant is served.
- `placeholder="blur"` : Improves perceived loading by showing a blurred version.

👉 These props make the above-the-fold image load immediately and optimally

Below the fold:

```
tsx
Copy code

<Image
  src="/product.jpg"
  alt="Product"
  width={400}
  height={400}
  loading="lazy" // ✅ delays loading until image is near viewport (default in Next.js for
  sizes="(max-width: 768px) 100vw, 33vw" // ✅ serve smaller images on mobile
  placeholder="blur" // optional: smoother UX for lazy-loaded images
/>
```

- `loading="lazy"` : Defers load until near viewport. (default in Next.js for all non-priority images).
- `sizes` : Lets Next.js pick the best size for the current viewport → improves bandwidth use.
- `placeholder="blur"` : Shows a placeholder before the actual image loads.

- How does nextjs calculate the image dimensions when image using aspect ratio
A: Uses Container width to determine the height reserved for the img eg if aspect (16 / 9) with 800px wide container, reserves $800 * (9/16) = 450\text{px}$ for height
- Is it always good to use font-display: swap?
A: NO, if font-display: swap then **flashes fallback font**, then browser 'swaps' in custom font. Improves perceived performance but intros FOIT/FOUT trade-offs:
FOIT = Flash of **Invisible** Text (what happens w/o swap)
FOUT = Flash of **Unstyled** Text (what happens with swap)
- Core Web Vitals vs Lighthouse test?
A: Core Web Vitals = **user-centered performance metrics** determined by Google. 3 main metrics (LCP, FID, CLS). Matters because directly affects SEO rankings. Shows actual UX across devices.
Lighthouse = **open-source auditing tool** for Performance, Accessibility, Best

Practices, SEO, PWA compliance. Great for debugging and finding potential issues. Provides actionable recommendations (eg defer offscreen imgs).

- Any cases where you shouldn't lazy-load content below the fold?
A: Critical user journeys/**primary user flow**, **smooth scrolling** experiences (preload a bit of below-the-fold stuff barely), **SEO-critical content** eg article body, product descriptions. While Google can crawl lazy loaded content, misconfigured lazy loading may prevent bots from seeing it.

Google can crawl lazy-loaded content, **but only if implemented correctly**. Can only load content that appears when scrolling if triggered by **standard browser APIs** (eg IntersectionObserver).

If lazy loads depend on user gestures (eg onScroll, onClick), google cant perform those, and it'll never render

- Why would u want bots to see your website content?
 1. **Index your pages** so Google can rank them
 2. **Rank properly** - Google use Core Web Vitals, page exp, and actual text/images to rank
 3. **Provide search snippets** - Featured snippets - rich results, previews rely on accessible content

If bots don't see, you will rank for far fewer keywords, competitors with visible content outrank you, and Google thinks your page is "thin content"

Images

☐ CDN

A CDN (Content Delivery Network) for images means images served from **edge servers** distributed around the world, rather than single origin server.

In Nextjs, directly integrated into next/image component:

- [Next.js](#) optimizes images (resizing, webp/avif conversion, compression)
- Images are cached and delivered via a CDN layer (vercel's edge network by default, or your configured CDN in [next.config.js](#) eg the remotePatterns to allow certain formats/sources)
- Users get image from nearest edge server, which improves load time and Core Web Vitals (esp LCP)

Where it would be used:

- ecomm sites product thumbnails -> served in mult sizes depending on device
- blogs/news with article hero images optimized for mobile vs desktop
- Social apps -> user uploaded imgs scaled to fit diff screen densities
- Marketing/landing pages -> hero banners need to load super fast worldwide

How to implement in Nextjs

1. Next/Images automatically served from Vercel's CDN, on self-hosted Nextjs, can config **external image CDN** in next.config.js
2. Configuring Remote Images

```
js Copy code  
  
// next.config.js  
module.exports = {  
  images: {  
    remotePatterns: [  
      {  
        protocol: 'https',  
        hostname: 'cdn.example.com',  
        pathname: '/images/**',  
      },  
    ],  
  },  
}
```

Lets you safely serve images from own CDN or third party providers like Cloudfare, Akamai, Imgix

Why use CDN:

- Faster LCP since images come from edge servers nearer to user
- Optimization built in from Nextjs
- Bandwidth savings since download smallest image variant they need
- Handles global traffic spikes better than single origin server

Why NOT to use CDN:

- **Internal apps** where all users are local, CDN overkill
- **Static-only small sites** where you can serve optimised images at build time eg info websites or shop websites
- If already using specialized service like Cloudinary/Imgix, u may let them handle instead of Nextjs

Examples:

- Product images optimized per device

☐ Format: WebP, SVG for icons

WebP: modern raster image format developed by Google. Supports lossy and lossless compression, transparency (like PNG) and animation (like GIF). It's 26%+ smaller in file size than JPEG/PNG for similar quality.

SVG (Scalable Vector Graphics): A vector-based image format defined in XML. SVGs don't store pixels but **shapes/paths** (they are a mathematical equation built w code), so they **scale w/o losing quality**. Best suited for icons, logos and illustrations.

In [Next.js](#), these formats can be used with the `<Image/>` component or imported directly as assets.

How to use in [Next.js](#):

WebP:

In NextJS just drop them in `/public` and import with `<Image src="hero-image.webp"/>`

SVG for Icons:

```
jsx Copy code

import Logo from '../public/logo.svg' // with next.config.js SVGR enabled

export default function Header() {
  return (
    <header>
      <Logo width={40} height={40} />
      <h1>App Name</h1>
    </header>
  )
}
```

Or inline:

```
jsx Copy code

const Icon = () => (
  <svg width="24" height="24" viewBox="0 0 24 24">
    <path d="..." />
  </svg>
);
```

Why choose webp and svg?

WebP advantages:

- Smaller file sizes -> faster page load -> better core web vitals
- Supports animation + transparency in 1 format
- Widely supported by modern browsers

SVG advantages:

- Infinitely scalable in size -> no pixelation on 4k displays
- Styleable with CSS/JS (colors, stroke weight, hover effects, animations since code)
- Great for UI icons, logos, charts

When NOT to use them:

WebP drawbacks:

- Older browsers don't support it (though polyfills (piece of code that provides functionality to older browsers)/Next.js fallback can help
- Sometimes slower to encode during build compared to PNG.

SVG drawbacks:

- Bad for complex, photo-like images since tons of math calcs = huge file sizes compared to WebP/JPEG
- Inline SVGs can bloat HTML if overused
- If loaded from untrusted source SVGs can carry XSS risks (since XML/JS capable)

Alternatives to consider:

- JPEG/PNG for legacy support or photography where encoding matters less
- AVIF: even smaller than webp in many cases, but slightly slower to encode/decode
- Icon fonts: generally replaced by SVGs due to accessibility and scalability

Questions:

- Does Next.js automatically turn your png to a webp? So I don't need to care about converting my pngs to webps when adding them to Image src?
- Does next.js automatically turn Image that would normally be webp in modern browser into png on old browser that wouldn't support webp?
- What are polyfills

☐ Priority-based loading, lazy loading via loading="lazy"

Priority-based loading (priority)

In Next.js, the `<Image/>` component has a priority prop.

- Tells Next.js "this image is critical for above-the-fold content; preload it ASAP"
- Usual applied to hero banners, logos, or key visuals to appear
- Next.js will auto generate `<link rel="preload">` for these

Lazy loading (loading="lazy")

- Next.js by default makes Images use lazy loading.
- Reduces initial page load and improves performance, esp for long, scroll-heavy pages.

loading="eager" = force loading immediately (rare)

Why choose:

- lazy loading cuts down number of requests at page load -> faster LCP

Don't overuse priority: only most critical 1-2 images should have it, otherwise negate

benefits

Questions:

- What is difference between fetchpriority=high, loading=eager, and priority

A:

fetchpriority="high" is relatively new HTML attribute that gives browser hint on how urgently image should be downloaded.

loading="eager" is HTML attribute tells browser to load image immediately, regardless of whether visible in viewport

priority is nextJS specific <Image/> prop, **combines** eager+preload+fetchpriority for **maximum priority**, sets these things under the hood:

- adds fetchpriority="high"
- adds <link rel="preload"> in the <head>
- forces eager loading

☐ Alt text

Alt text is descriptive text added to images through the alt attribute . In NextJS, you use <Image> component from next/image which requires an alt prop.

Purpose:

- Accessibility for screen readers, which read the text aloud for visually impaired users
- SEO, search engines use it to understand what image represents
- Fallback if image fails to load, alt text is displayed

Where it's used:

- Marketing pages - Hero banners, product mockups, customer logos
- E-comm - product images with descriptive alt text for accessibility and SEO
- Social apps - profile pictures or posts (twitter and instagram both use alt text)
- Dashboards & internal tools - charts & graphs with meaningful descriptions.

For **decorative images**, use alt="" (empty string) so screen readers skip.

Good vs Bad alt text:

- Describe in 5-15 words
- Convey **purpose** of image, not just its literal content
- Avoid "picture of"
- If image failed to load, would alt text describe it?

☐ Responsive Images: srcset, <picture>

Responsive images automatically serve different image sizes depending on user's device (mobile, tablet, desktop, etc)

Traditionally, in HTML, you'd do this with srcset (provides multiple image sources at different resolutions) and <picture> (lets you serve entirely different images (eg cropped mobile version vs wide desktop version)).

In NextJS, next/image generates srcset and <picture> automatically when you define widths/sizes.

Implementation in NextJS:

```
jsx Copy code

import Image from 'next/image';

export default function Hero() {
  return (
    <Image
      src="/hero.jpg"
      alt="A marketer analyzing campaign data"
      width={1200}
      height={600}
      sizes="(max-width: 768px) 100vw, (max-width: 1200px) 50vw, 1200px"
      priority
    />
  );
}
```

sizes: Tells browser how much space image will take at different breakpoints.

"(max-width: 768px) 100vw, (max-width: 1200px) 50vw, 1200px"

- NextJS generates multiple versions (srcset) automatically and browser picks the best one.

Using <picture> for Art Direction

```
jsx Copy code

<picture>
  <source media="(max-width: 768px)" srcSet="/hero-mobile.jpg" />
  <source media="(max-width: 1200px)" srcSet="/hero-tablet.jpg" />
  
</picture>
```

In NextJS, you'd only need <picture> if you want different artwork, since <Image> already handles responsive resizing for you.

❑ <link rel="preload">

<link rel="preload"> is a **resource hint** that tells the browser to start fetching an asset early - before it normally would.

Use for hero, above-the-fold images.

In next/image, just set priority on a <Image> which adds a preload hint

You can **Manual Preload** in <Head> if you need to preload an image not rendered immediately (like background CSS images) you can add:

jsx

Copy code

```
import Head from 'next/head';

export default function CustomPreload() {
  return (
    <Head>
      <link
        rel="preload"
        as="image"
        href="/background.jpg"
        imagesrcset="/background@1x.jpg 1x, /background@2x.jpg 2x"
        imagesizes="100vw"
      />
    </Head>
  );
}
```



❑ Adaptive loading (network speed, screen size)

See [Adaptive Loading](#) for performance.

In NextJS, you can do Adaptive Loading by Network Speed for images with the Network Information API (navigator.connection).

jsx

 Copy code

```
import Image from 'next/image';
import { useEffect, useState } from 'react';

export default function AdaptiveImage() {
  const [quality, setQuality] = useState(75);

  useEffect(() => {
    if (navigator.connection && navigator.connection.effectiveType.include
      'slow-2g') {
      setQuality(30); // lower quality for slow networks
    }
  }, []);

  return (
    <Image
      src="/product.jpg"
      alt="Smartwatch product image"
      width={800}
      height={800}
      quality={quality}
    />
  );
}
```

Use the quality prop in Image to do this.

Accessibility

☐ Keyboard interactions and shortcuts

Keyboard interactions ensure users can navigate and use your app **without a mouse**, using only the keyboard (Tab, Enter, Space, Arrow keys, Escape, etc.)

Keyboard shortcuts allow power users to trigger specific actions quickly eg ctrl+k to open search.

In NextJS,

1. Ensure interactive elements follow **ARIA accessibility standards** (topic below).
2. Add custom event handlers (onKeyDown, onKeyUp) for custom components
3. Providing shortcuts in a way that doesn't break accessibility

Implementation in NextJS:

1. Accessible Navigation with Tab+Enter

jsx

 Copy code

```
export default function AccessibleButton() {
  return (
    <button onClick={() => alert("Clicked!")}>
      Press Enter or Space to activate
    </button>
  );
}
```

Native `<button>` already supports keyboard activation (Enter, Space)
Avoid `<div onClick>`, unless you handle keyboard events and ARIA roles

2. Custom Keyboard Handling

If you build a custom component (like a menu), you must add key handling manually.

jsx

 Copy code

```
import { useState } from "react";

export default function Dropdown() {
  const [open, setOpen] = useState(false);

  const handleKeyDown = (e) => {
    if (e.key === "Enter" || e.key === " ") setOpen(!open);
    if (e.key === "Escape") setOpen(false);
  };

  return (
    <div
      role="button"
      tabIndex={0} // makes div focusable
      onClick={() => setOpen(!open)}
      onKeyDown={handleKeyDown}
    >
      {open ? "▲ Menu Open" : "▼ Menu Closed"}
    </div>
  );
}
```

sdadasdasdasdfasdasdasfasdasf

- ☐ WAI-ARIA roles, states, properties: Semantic HTML, img alt text, ARIA attributes, role attribute
- ☐ Headings hierarchy
- ☐ Focus management: focus indicators, tab order
- ☐ Visual: Color contrast, font size

SEO

- ☐ <title>, <meta>
- ☐ Sitemap
- ☐ JSON structured data
- ☐ Semantic markup
- ☐ Heading hierarchy
- ☐ SSR or SSG (Pre-generate pages)

User Experience

- ☐ States: Loading, error, success, empty, offline
- ☐ Error handling: ignore, retry, or display?
- ☐ Infinite scrolling
- ☐ Long strings
- ☐ Mobile-friendliness

Security

- ☐ HTTPS
- ☐ XSS
- ☐ CSRF
- ☐ CORS
- ☐ CSP

i18n

- ☐ <html lang>, hreflang attribute

- ☐ Right-to-left
- ☐ Use template strings
- ☐ Dates, currencies, numbers formatting
- ☐ Avoid text in images
- ☐ Input method editors (IMEs)

Common Patterns

- ☐ Infinite scrolling
- ☐ Event sourcing
- ☐ Reducer pattern / Flux architecture
- ☐ Undo/redo
- ☐ Media streaming
- ☐ Conflict resolution
- ☐ Offline-first/usage

Extra

Database/Scalable Systems

Atomicity

- All transactions either fail or succeed so database is never halfway complete

Query performance

Long-running tasks

Refer to operations that take a significant amount of time to complete, such as:

- Exporting millions of db records
- Processing video files
- Generating complex reports
- Running ML model inference
- Sending thousands of emails in a campaign

In Next.js, long-running tasks should NOT be handled inside the main request/response cycle (eg normal API route or server action).

Why? Because HTTP requests time out (after 30-60s depending on server) and it will block your server resources -> bad for scalability and UX.

Instead, you offload these jobs to **background processes** or **external task queues**.

How it's implemented:

- Background jobs (queue-based): API triggers a job, push it to a queue, worker picks it up then processes it separately (Resend, BullMQ, Upstash Redis (queues), Temporal.io)
- Async processing with Webhooks: API triggers another service -> webhook callback when done (Stripe & Supabase webhooks)
- Serverless schedules jobs (cron): scheduled background task (eg clean old data), Vercel Cron Jobs, GitHub Actions
- Third-party compute services: offload to a compute provider for heavy tasks (AWS lambda, google cloud functions, vercel edge functions).

Examples of real world features:

- Sending a batch of 100k marketing emails -> user triggers it -> next.js api route queues the job -> worker processes and sends asynchronously
- Exporting large CSV or PDF reports -> API triggers -> system processes -> sends email or notif when ready

What are workers:

Separate program or serverless function whose only job is to listen for new jobs on the queue and process them.

Authentication/Security

API Authentication

Proving who is making a request to your API routes (or server actions) - and deciding whether you should let them do it.

You must verify that:

- They are who they claim to be (authentication)
- They are allowed to do what they're trying (authorization, slightly separate)

Where API auth is used:

- Fetching user-specific dashboards

Typical implementations:

- Sessions (stateful) API reads cookies (like auth_token) and validates against a database session, good for APIs that must scale horizontally with built-in csrf protection
- JWT tokens (stateless) API checks for a signed token in the authorization headers or cookies, good for APIs that must scale horizontally

- API keys, each request carries a unique API Key in headers, verified serversides, good for systems where you trust the user less (exposed to 3rd parties, bots)
- OAuth tokens (for user services), bearer token issued by Google, GitHub, etc, integration with 3rd party platforms

Permissioning

Protecting sensitive information

Prompt

As a senior frontend developer, optimize this Next.js TypeScript App Router code using best practices. Focus on performance improvements, Lighthouse optimization, code splitting, caching strategies, and server-client responsibility separation. Ensure efficient API handling, reduce unnecessary re-renders, and implement best practices for maintainability and scalability. Use modern Next.js features like React Server Components, Suspense, and optimized data fetching where applicable

"As a senior frontend developer, optimize this Next.js TypeScript App Router code using best practices. Focus on:

- **Performance:** Improve Lighthouse scores, reduce bundle size, optimize rendering, and minimize re-renders.
- **Code Splitting & Lazy Loading:** Implement dynamic imports (`next/dynamic`) where applicable to reduce initial load time.
- **Server Components & Suspense:** Use React Server Components (RSC) and Suspense where applicable for efficient data fetching.
- **Data Fetching:** Optimize API calls using `fetch` inside Server Components, `useQuery` (if using React Query), or parallel fetching via `Promise.all`.
- **Edge Functions & Caching:** Leverage **Edge Runtime**, ISR (Incremental Static Regeneration), and `revalidate` for caching where needed.
- **State Management:** Use server state instead of unnecessary client-side state when possible, and optimize `useState/useEffect` usage.

- **Accessibility & SEO:** Improve accessibility (`aria-*` attributes, proper semantics) and SEO (`next/head`, metadata).
- **Security:** Ensure secure API handling, CSRF protection, and environment variable management.
- **Scalability & Maintainability:** Improve code structure, type safety with TypeScript, and ensure modular, reusable components. "*"